

Voici une **synthèse complète, précise et non ambiguë** de tout ce que nous avons décidé dans ce chat. Je vérifie en même temps la cohérence globale et j'indique les éventuels points restant à définir.

1. Objectif de l'application

Apprentissage d'une langue B pour un utilisateur natif de la langue A.

L'utilisateur a des **sessions quotidiennes d'apprentissage** composées :

- d'exercices sur les **mots** ;
- d'exercices sur les **phrases** (compréhension, traduction, écoute, prononciation) ;
- plus tard : éventuellement des sessions mixtes.

L'idée maîtresse : **ne pas multiplier les fonctionnalités** mais rendre 1–2 catégories parfaitement solides.

2. Deux piliers fondamentaux

Nous avons convergé vers une architecture propre, simple et robuste :

2.1. WordSession

Session dédiée au vocabulaire isolé.

- Exercices :
 - traduction mot B → A
 - (optionnel plus tard) traduction A → B
- Score : **SRS individuel par mot.**
- Objectif : apprendre et réviser le lexique.
- Effet anti-priming naturel, car séparé des phrases.

2.2. SentenceSession

Session dédiée aux phrases, au contexte et à la grammaire.

- Exercices :
 - phrase B → A (traduction ou compréhension globale)
 - plus tard : phrase A → B
 - écoute (dictée : entendre la phrase, la réécrire)
 - prononciation (répéter la phrase)
- Score :
 - **SRS par groupe de phrases** (pattern grammatical + thème lexical)

- À l'intérieur d'un groupe :
 - **seenCount** par phrase
 - **score local** par phrase (Good=1, Ok=0.5, Again=0)
- Objectif :
 - renforcer la grammaire
 - apprendre en contexte
 - varier les exemples
 - éviter la répétition excessive d'une même phrase.

3. Architecture du contenu

Tout le contenu linguistique est structuré autour de **catégories thématiques**, proches de Duolingo :

- animaux
- vêtements
- météo
- maison
- etc.

Chaque catégorie contient :

1. **Liste de mots** (avec traduction + tags lexicaux)
2. **Liste de phrases** (pré-écrites ou tirées d'un corpus)
3. **Tags grammaticaux** associés (simple, présent, be+adj, etc.)
4. **Groupe de phrases** (pattern grammaticaux + thème)

La progression est donc :

Catégorie → mots → phrases → écoute/prononciation.

PROF

Tu introduis la grammaire **progressivement**, comme un tutoriel.

4. Problèmes identifiés et solutions adoptées

4.1. Génération automatique de phrases = abandonnée

Pour des raisons de :

- cohérence grammaticale
- naturalité
- pédagogie
- maintenance
- qualité du contenu

Tu utilises des phrases **pré-écrites** ou issues de corpus libres (Tatoeba, OPUS).

4.2. Répétition des phrases : problème détecté chez Duolingo

Solution adoptée :

- tirer les phrases **sans remplacement** dans une session ;
 - imposer un **cooldown** interne pour éviter qu'une phrase déjà vue réapparaisse trop vite ;
 - sélectionner dans un groupe la phrase la **moins vue** (**seenCount** **minimal**) ;
 - viser que chaque phrase soit vue **1 à 3 fois maximum** dans toute la vie utilisateur.
-

4.3. Problème du priming (voir "dog" dans une phrase puis carte "dog")

Solution :

- séparation complète WordSession / SentenceSession
 - (plus tard) règles anti-priming internes si tu fais des sessions mixtes.
-

4.4. Score des phrases : pas de SRS individuel par phrase

Bonne décision pour éviter dilution du signal SRS.

Score final adopté :

Niveau groupe

- SRS complet (SM2 modifié)
- Contrôle la cadence de révision du pattern grammatical

Niveau phrase

- **seenCount** = pour distribuer l'exposition
 - **localScore** = pour sélectionner les phrases moins réussies
-

5. Fonctionnalités retenues dans l'application

5.1. Fonctionnalités "Mots"

- Traduction B → A
 - (optionnel après MVP) traduction A → B
 - Visualisation des définitions et exemples
 - SRS individuel
 - Statistiques : score global du vocabulaire
-

5.2. Fonctionnalités "Phrases"

Avec contenu pré-écrit :

- Traduction B → A
 - (plus tard) traduction A → B
 - Exercices d'écoute (entendre → écrire)
 - Exercices de prononciation (répéter)
 - Score :
 - SRS par groupe
 - sélection intelligente des phrases via seenCount/localScore
 - Statistiques : score global des phrases
-

5.3. Fonctions transversales

- Antipriming via séparation des sessions (et plus tard via des règles internes)
 - Déblocage progressif de la grammaire
 - Progrès par catégories (chapitres)
 - Sessions quotidiennes avec ordre recommandé :
 1. WordSession (révisions)
 2. SentenceSession (contexte)
 - Système de stats globales :
 - mots
 - phrases
 - écoute
 - prononciation
-

6. Fonctionnalités repoussées (non-MVP)

- Prononciation (nécessite Speech-to-Text fiable)
 - Génération automatique de phrases
 - Sessions mixtes mots + phrases (possible plus tard)
 - SRS individuels par phrase
 - Grammaire avancée
-

7. Ce qui reste éventuellement à définir

Rien d'essentiel n'est oublié, mais il reste quelques points à fixer quand tu commenceras à implémenter :

1. Format exact des données (JSON ? SQLite ? tags ?)
2. Nombre optimal de phrases par groupe (moi je recommande **8 à 20**).
3. Calibrage exact du SRS des groupes (intervals initiaux, ease factor).
4. Choix pour la traduction $A \rightarrow B$ (inclusion ou non au MVP).
5. Corpus initial (Tatoeba filtré est le candidat premier).
6. Ordonnement précis d'une session (ex : 8 mots puis 4 phrases).

Mais tout cela ne change pas la **structure conceptuelle**, qui elle est complète.

Conclusion

Voici l'architecture finale de ton application :

- **WordSession (SRS par mot)**
- **SentenceSession (SRS par groupe + score local par phrase)**
- Séparation stricte pour éviter le priming
- Variété forte des phrases (seenCount minimal)
- Catégories thématiques + progression de la grammaire
- Écoute et prononciation rattachées aux phrases
- Corpus pré-écrit (pas de génération aléatoire)
- Système de stats clair : mots, phrases, écoute, prononciation

Cette architecture est solide, réaliste, scalable et pédagogiquement cohérente.

Si tu veux ensuite, on peut définir ensemble :

- le format exact d'un groupe de phrases,
- la structure d'une catégorie complète,
- la pipeline pour importer ton premier corpus,
- ou le design d'une session complète du point de vue utilisateur.

Quel point veux-tu détailler maintenant ?

Oui, clairement.

Tu as bien défini les **sessions** (WordSession, SentenceSession), mais une application sérieuse a aussi des **fonctionnalités hors session** essentielles pour l'apprentissage, la motivation, la gestion du contenu et l'expérience utilisateur.

Je te liste **toutes les fonctionnalités importantes hors session**, puis je distingue celles qui sont **indispensables, optionnelles, ou futures**.

1. Fonctionnalités indispensables (MVP réel)

1.1. Page “Progression / Stats”

Elle doit afficher clairement :

- score vocabulaire (moyenne SRS des mots)
- score phrases (SRS des groupes + stats locales)
- score écoute
- score prononciation (plus tard)
- nombre de mots connus
- nombre de phrases exposées
- temps d'étude total
- streak / régularité

Rôle : feedback immédiat, motivation, pilotage du plan d'apprentissage.

Tu ne peux pas faire une app crédible sans un système de stats.

1.2. Page “Vocabulaire” (liste des mots)

Permet de :

- voir tous les mots appris
- voir leur état SRS (intervalle, nextReview, réussite)
- cliquer un mot → ouvrir une fiche détaillée :
 - définition
 - exemples (phrases où il apparaît)
 - prononciation
 - tags (nom, verbe, adjectif, thème)

Rôle : donner au user le contrôle de ce qu'il apprend.

1.3. Page “Phrases / Grammaire”

Même principe que la page vocabulaire mais pour les phrases et les patterns grammaticaux :

- liste des groupes de phrases (patterns)
- pour chaque groupe :
 - explication de la règle

- liste des phrases disponibles
- phrase → détail (traduction, prononciation, mots inclus)

Rôle : indispensable pédagogiquement.

Le user veut comprendre *pourquoi* la phrase a telle forme.

1.4. Système de “chapitres” / “thèmes”

Tu as déjà posé ce concept.

Il faut une interface :

- liste des catégories de vocabulaire
- barres de progression
- verrouillage/débloccage progressif
- accès rapide aux exercices associés

Rôle : structurer l'apprentissage (sinon le user est perdu).

1.5. Paramètres utilisateur

Au minimum :

- objectif quotidien (ex : 5–10–15 minutes)
- langue A et langue B
- heure de rappel (notification quotidienne)
- variété vs répétition (paramètre avancé)

Insignifiant techniquement mais obligatoire côté UX.

2. Fonctionnalités optionnelles (valeur ajoutée mais pas vitales au MVP)

2.1. Mode “révision libre”

En dehors des sessions du jour :

- réviser n'importe quel mot
- réviser n'importe quel groupe de phrases
- refaire les phrases avec mauvais score local
- écouter des phrases aléatoires
- cartes aléatoires “pour s'amuser”

Utile pour les utilisateurs actifs.

2.2. Mode “lecture”

Tu génères (ou sélectionnes) un petit texte simple contenant des mots que l'utilisateur connaît déjà.

- très puissant pour le transfert lexique → compréhension
- demande une logique de filtrage du vocabulaire connu
- niveau débutant uniquement (phrases courtes)

Pas essentiel, mais très utile plus tard.

2.3. Mode “cartes favorites”

Le user peut “épingler” des mots/frases difficiles, et les revoir hors SRS.

2.4. Notifications intelligentes

Plutôt que des rappels bêtes, tu peux envoyer :

- “Tu as 5 mots à revoir”
 - “Un groupe grammatical est dû pour révision”
-

2.5. Analyse d'erreurs

Le système pourrait afficher :

- types d'erreurs fréquentes (verbes, adjectifs, gender)
- mots toujours ratés
- phrases problématiques

Très formateur, mais nécessite un peu de logique statistique.

PROF

3. Fonctionnalités futures (hors MVP mais très utiles pour scalabilité)

3.1. Création de contenu collaborative

Un éditeur interne :

- ajouter/changer des mots
- ajouter des phrases
- ajouter des explications grammaticales
- associer des tags
- modération communautaire

Si ton app décolle, c'est un “must”.

3.2. Mode “production libre” (écriture)

Par exemple, le user écrit une phrase dans la langue B, et l’application propose :

- corrections
- réécriture naturelle
- classification grammaticale
- exemples similaires

Demande de l’IA → donc hors MVP.

3.3. Mode “conversation guidée”

Exercice interactif semi-libre :

L’appli pose des questions → le user répond → analyse.

Tu peux utiliser du pattern-matching simple (pas forcément IA).

3.4. Mode “entraînement intensif SRS”

Une session 100% basée sur la dette SRS, sans catégories.

4. Fonctionnalités nécessaires à long terme pour une vraie application

Je les liste pour être exhaustif, mais tu peux les ignorer jusqu’à ce que ton MVP soit solide.

4.1. Sauvegarde cloud (multi-machine)

- synchroniser le SRS, le vocabulaire, les stats
- indispensable à terme

4.2. Comptes utilisateurs (authentification)

- email
- Google/Apple login
- anonymes temporaires

4.3. Export / import des données utilisateur

Pour les power users.

5. Vérification : As-tu oublié quelque chose ?

- Non, tu n'as rien oublié du cœur pédagogique.
- Tu n'as rien oublié des exercices essentiels.
- Tu n'as rien oublié des risques techniques (priming, génération, répétition).
- Tu n'as rien oublié de l'architecture SRS (mots individuels, groupes de phrases, scores locaux).

Les seules choses manquantes concernaient :

- l'interface hors-sessions
- la gestion du contenu
- la progression et les stats
- l'accès aux données (listes, catégories)

Je viens de les ajouter de manière exhaustive.

Synthèse ultra-courte

En dehors des sessions, ton app doit absolument fournir :

1. **Progression (stats claires)**
2. **Listes de mots et phrases avec détails**
3. **Catégories / chapitres / progression**
4. **Paramètres utilisateur**
5. (Optionnel) révision libre, textes, favoris
6. (Plus tard) création de contenu, écriture, conversation, sync cloud

Si tu veux, je peux maintenant :

- définir exactement les écrans non-SRS (UI/UX minimal),
- proposer une structure de navigation (flow complet de l'app),
- ou déterminer ce qui doit absolument être dans le MVP.